UltraXAI — Response to Methodology Review

Answers to "Methodology Gaps and Required Developer Actions"
Developed by: Abdul Rafay — Final Year Project (FYP)
Date: 25 October 2026

This report answers each of the 9 review items. For every item it says what was changed in the code, what the code actually does now, and what is still pending. No accuracy numbers are reported here: final numbers will be added only after the training run on the local GPU finishes and the test set has been evaluated.

#	Review item	Status
1	Dataset source, composition, preprocessing	**DONE**
2	Segmentation split bug	**DONE** (code); **NEEDS GPU RUN** (re-train)
3	Final prediction task and labels	**DONE**
4	Model names vs. code	**DONE**
5	Training configuration and reproducibility	**DONE** (saved automatically)
6	Held-out test evaluation	**DONE** (script); **NEEDS GPU RUN** (numbers)
7	Explainability	**PARTLY DONE** — Grad-CAM++ yes, SHAP **NOT IMPLEMENTED**
8	End-to-end workflow	**DONE**
9	Deliverables	**PARTLY DONE**

1. Dataset source, composition and preprocessing — DONE

Dataset actually used: BUSI (Breast Ultrasound Images Dataset), Al-Dhabyani W., Gomaa M., Khaled H., Fahmy A., *Data in Brief* 28 (2020) 104863, doi:10.1016/j.dib.2019.104863. Downloaded from Kaggle (aryashah2k/breast-ultrasound-images-dataset) with `download_busi.py`. BUS-BRA, UDIAT, STU and OASBUD are **proposed only** and were not used in any experiment.

Class	Images in BUSI	Used for classification	Split train / val / test
Benign	437	Yes	305 / 66 / 66
Malignant	210	Yes	147 / 31 / 32
Normal	133	No (excluded, see item 3)	—
Total	**780**	**647**	**452 / 97 / 98**

Masks: BUSI gives a mask file for each lesion image (`name_mask.png`). Masks are resized with nearest-neighbour interpolation and binarised at a threshold of 128. Images without a mask are skipped for segmentation. Some BUSI images have extra masks (`_mask_1`); only the first mask is used.

Preprocessing as implemented: images are read in grayscale. Classification: resize to 224×224, the grayscale channel is copied to 3 channels, ImageNet normalisation (mean 0.485/0.456/0.406, std 0.229/0.224/0.225). Segmentation: resize to 256×256, normalised with mean 0.485, std 0.229. Augmentation is applied to training images only (flips, affine, elastic/grid distortion, speckle and Gaussian noise, blur, brightness/contrast, gamma, CLAHE, sharpen, coarse dropout). No images were removed for quality reasons.

Augmentation volume: 452 training images + 2,260 augmented copies = **2,712 training images**. Validation and test images are never augmented.

2. Segmentation train / validation / test split — DONE (code), NEEDS GPU RUN

Bug confirmed. `train_attention_unet.py` created a split but then loaded *all* images for both the training and validation datasets, so validation Dice was measured on training images.

Fix: `BreastUltrasoundDataset` now accepts an explicit `samples` list. The training dataset gets only `splits["train"]` and the validation dataset gets only `splits["val"]`. The test partition is not loaded during training. The split is stratified at image level with seed 42 (BUSI has no patient IDs, so a patient-level split is not possible). No augmentation is applied before splitting, so augmented copies cannot leak into validation or test.

Manifest: the exact file list of each partition is saved to `unet_split_manifest.json` (and `classifier_split_manifest.json` for the classifiers) so the split can be audited.

Verified: on a 40-image synthetic test set, training now loads 28 images and validation 6, with 6 held out for test. Before the fix both loaded all 40.

Pending: the U-Net must be re-trained with the fixed split, and its logs added. Any old U-Net validation Dice is not reliable.

3. Final prediction task and class labels — DONE

Final task (one sentence): the classifier is **binary — Benign (0) vs. Malignant (1)**, trained on lesion images only.

Normal images are excluded from the classifier because they contain no lesion to classify. In the full pipeline, a normal scan is handled before the classifier: if YOLO finds no lesion and the Attention U-Net segments fewer than 10 lesion pixels, the system skips the classifier and reports "No tumor pixels segmented — Normal / Benign". The code, README and this report all use the binary task. The older 3-class confusion-matrix image in the repository is from an earlier experiment and is not part of the final design.

4. Model architecture names and actual pipeline — DONE

Component	Exact implementation in code
Detection	Ultralytics YOLOv8m-seg (base yolov8m-seg.pt), input 640, confidence 0.25
Segmentation	Attention U-Net (custom, training/models/attention_unet.py), 1 input channel, 1 output channel, input 256×256, Dice + BCE loss (0.5 / 0.5)
Classifier 1	torchvision ResNet50, ImageNet IMAGENET1K_V2 weights, custom head, 2 classes
Classifier 2	torchvision **EfficientNetV2-S**, ImageNet IMAGENET1K_V1 weights, custom head, 2 classes
Ensemble	Weighted average of probabilities: 0.40 × ResNet50 + 0.60 × EfficientNetV2-S

Correction: earlier documents said "EfficientNetV2-B0". The code uses **EfficientNetV2-S**. The class name `EfficientNetV2B0Classifier` is historical and is kept so existing code does not break; its docstring and the README now say V2-S. Models and datasets listed only in the proposal are future work, not implemented.

5. Training configuration and reproducibility — DONE

Every classifier run now saves `backend/weights/classifier_run_config.json` automatically. It records the exact launch command, all arguments, train/val counts, model variants, optimizer, scheduler, loss, early-stopping rule, and the environment (Python, PyTorch, CUDA, cuDNN, GPU name and memory, OS).

Setting (classifiers, final run)	Value
Command	python run_full_training.py --data_dir datasets/BUSI --copies 5 --epochs 80
Epochs / batch / image size	80 (max) / 16 / 224×224
Optimizer	AdamW, lr 1e-4, weight decay 1e-4, gradient clipping 1.0
Scheduler	3-epoch linear warmup, then cosine decay

Setting (classifiers, final run)	Value
Loss	Class-weighted cross-entropy, label smoothing 0.1, mixup $\alpha = 0.2$
Early stopping	Stop after 15 epochs without validation F1 improvement; keep best-F1 weights
Seed	42
Train / val / test	2,712 (452 original + 2,260 augmented) / 97 / 98
Software (known)	PyTorch 2.6.0 + CUDA 12.4, Python 3.12, Albumentations 2.0.8, Windows

Pending: GPU model, GPU memory and training time will be filled from the saved config file after the run finishes.

6. Test-set evaluation and evidence — DONE (script), NEEDS GPU RUN

New script `training/evaluate_test.py`, run automatically as step 4 of `run_full_training.py`. It runs **once** on the untouched 98-image test split after model selection. It reports ResNet50, EfficientNetV2-S and the ensemble separately: accuracy, precision, sensitivity (recall), specificity, F1, NPV, ROC-AUC, the confusion matrix, and the test sample count per class.

Outputs: `evaluation/results/test_metrics.json` and `confusion_matrix_*.png`. For segmentation, Dice and IoU are already computed during validation (`utils/metrics.py`).

No numbers are reported yet. They will be taken directly from `test_metrics.json` after the GPU run. No invented or estimated values are used.

7. Explainability (Grad-CAM++ and SHAP) — PARTLY DONE

Grad-CAM++ (implemented): `training/utils/explainability.py`. The backend applies it to the **ResNet50** branch, target layer `backbone.layer4[-1]` (the last convolution block), for the **predicted class**. The heatmap is overlaid on the segmented region of interest and shown in the dashboard. Code also exists for EfficientNetV2-S (`features[-1]`), but the live pipeline shows ResNet50 only.

SHAP: not implemented. No SHAP code exists in the project. SHAP must either be implemented (for example a GradientExplainer on the ensemble with ~50 training images as background data) or removed from the thesis and listed as future work.

Assessment: no clinical or expert validation of the explanations has been done, and none is claimed. Assessment is qualitative: on correctly classified test images, check visually whether the heatmap falls on the lesion area shown by the ground-truth mask.

8. Final end-to-end workflow and integration — DONE

YOLO, Attention U-Net, the classifier ensemble and Grad-CAM++ all run together in one executable pipeline (`backend/app/services/pipeline_orchestrator.py`), served by FastAPI and shown in the React dashboard.

Step	Input → Output
1. Upload	Ultrasound image → grayscale array
2. YOLOv8m-seg	Full image → lesion box + crop (ROI). If nothing is found, the full image is used.
3. Attention U-Net	ROI → lesion mask. If YOLO missed but the mask has > 50 pixels, the mask box (+15% padding) is used as the ROI.
4. Decision	If the mask has < 10 lesion pixels → "Normal / Benign", classifier skipped
5. Ensemble	Segmented ROI (224×224) → Benign / Malignant, confidence, trust level (HIGH ≥ 0.90, MEDIUM ≥ 0.70)
6. Grad-CAM++	ResNet50, predicted class → heatmap overlay

Step	Input → Output
7. Dashboard	Diagnosis, confidence, box, mask, heatmap

Note: the classifiers are trained on full BUSI images, but in the pipeline they receive a cropped ROI. This mismatch should be stated in the thesis, or the classifiers should be re-trained on crops.

9. Deliverables — PARTLY DONE

Requested	Status
(a) Corrected segmentation split code	**DONE**
(b) Dataset / split manifest	**DONE** — saved as JSON on every run
(c) Model / config files	**DONE** — classifier_run_config.json
(d) Training and validation logs	**NEEDS GPU RUN**
(e) Trained checkpoints	**NEEDS GPU RUN** — backend/weights/
(f) Held-out test evaluation	**NEEDS GPU RUN** — script ready
(g) Grad-CAM++ / SHAP details	**PARTLY DONE** — Grad-CAM++ documented, SHAP missing
(h) Updated README / architecture	**DONE**
(i) Environment / hardware	**DONE** — recorded automatically

Next Steps

1. Run `python run_full_training.py --data_dir datasets/BUSI --copies 5 --epochs 80` (training + test evaluation), and save the console log.

2. Re-train the Attention U-Net with the fixed split: `cd training && python train_attention_unet.py --data_dirs ../datasets/BUSI --output_dir ../backend/weights`

3. Decide on SHAP: implement it or move it to future work.

4. Copy the numbers from `evaluation/results/test_metrics.json` and the config files into the thesis.

UltraXAI — Developed by Abdul Rafay